Assessment – 1

Section A — Concept Application

Question: Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

Recommended Development Methodology: Agile Methodology

For this project, I would recommend **Agile development methodology** because it supports **continuous improvement, flexibility, and frequent feedback**. In Agile, the project is divided into small iterations (sprints), allowing the team to develop, test, and deliver features step by step. It is suitable when requirements may change during development and when regular communication with stakeholders is required.

Reasons for choosing Agile:

1. **Flexibility to handle changing requirements:** The team can easily modify features based on user feedback.
2. **Continuous testing and improvement:** Bugs can be identified and fixed early in each sprint.
3. **Faster delivery:** Working modules can be released incrementally instead of waiting until the entire project is completed.
4. **Better collaboration:** Developers, testers, and customers can communicate regularly.

Risks of Choosing the Wrong Methodology:

1. **Increased project failure risk**
    - If a rigid methodology like Waterfall is chosen for a project with changing requirements, late changes can become expensive and may delay the project.
2. **Poor product quality and customer dissatisfaction**
    - Without regular feedback and testing, the final product may not meet user expectations, leading to rework, increased cost, and an unsuccessful project.

Conclusion:
Agile is the most appropriate methodology because it provides adaptability, faster delivery, and better alignment with user needs.

Question: Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

Git Workflow

To prevent code conflicts, accidental overwrites, and loss of work, the team should adopt a **Feature Branch Git Workflow**.

In this workflow, each developer creates a separate branch for their task instead of directly modifying the main branch. After completing the work, the changes are reviewed through a **pull request (PR)** and merged into the main branch only after approval and testing.

Git Workflow Steps:

1. **Create a new branch for each feature or bug fix**
    - Developers should not work directly on the main branch.
    - Example: feature/login-page or bugfix/payment-error
2. **Make regular commits**
    - Developers should save changes frequently with meaningful commit messages.
3. **Push changes to the remote repository**
    - Team members upload their branch so others can review and collaborate.
4. **Create a Pull Request**
    - Another team member reviews the code before merging it into the main branch.
5. **Pull latest changes before starting work**
    - Developers should update their local repository to avoid conflicts.

Important Git Commands:

1. **git clone**

- Used to create a local copy of a remote repository.
- It allows team members to download the project and start working on it.

Example:

git clone repository_url

2. **git commit**
    - Used to save changes locally with a meaningful message.
    - It creates a history of project changes and makes it easier to track problems.

    Example:

    git commit -m "Added user login feature"

3. **git pull**
    - Used to download the latest changes from the remote repository and merge them into the local branch.
    - Helps avoid conflicts caused by outdated code.

    Example:

    git pull origin main

4. **git push**
    - Used to upload local commits to the remote repository.
    - Allows other team members to access the latest work.

    Example:

    git push origin feature-branch

Question: Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the

above-average days — and explain why both steps cannot be done in a single loop.

Why Array is the Correct Data Structure:

An **array** is the correct data structure for storing 30 days of data because it allows multiple values of the same type to be stored under a single variable name. Instead of creating 30 separate variables (such as day1, day2, day3, etc.), an array stores all values in an organized way and allows easy processing using loops.

Advantages of using an array:

1. **Easy data management**
    - All 30 daily values are stored together and can be accessed using an index.
2. **Reduces code complexity**
    - A single loop can process all elements instead of writing separate statements for each variable.
3. **Efficient calculations**
    - Arrays make it easier to calculate totals, averages, search values, and compare data.

Example:

int hours[30];

Here, hours[0] stores the first day's data and hours[29] stores the 30th day's data.

Two-Step Program Logic:

Step 1: Calculate the Average

- Traverse the entire array using a loop.
- Add all 30 values to calculate the total.
- Divide the total by 30 to find the average.

Formula:

Average=Sum of all 30 days30Average = \frac{Sum\ of\ all\ 30\ days}{30}Average=30Sum of all 30 days

Example logic:

```
sum = 0;
for(i = 0; i < 30; i++)
{
   sum = sum + hours[i];
}
average = sum / 30;
```

Step 2: Identify Above-Average Days

- Traverse the array again.
- Compare each day's value with the calculated average.
- If a value is greater than the average, display that day.

Example logic:

```
for(i = 0; i < 30; i++)
{
   if(hours[i] > average)
   {
      printf("Day %d is above average", i+1);
   }
}
```

Why Both Steps Cannot Be Done in a Single Loop:

Both operations cannot be completed in one loop because the **average value is not known until all 30 values have been processed**.

- During the first loop, the program only calculates the total sum.
- To check whether a day is above average, the program needs the final average value.

- The average can only be calculated after the complete array has been read.

Therefore, the program requires **two separate loops**:

1. First loop → Calculate total and average.
2. Second loop → Compare values with average and find above-average days.

This approach is simple, efficient, and ensures accurate results.

Question: Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Why the Program Crashes on an Empty String Input:

A program that processes a string using a pointer may crash on an empty string because it tries to **dereference a pointer that does not point to a valid character**.

For example:

```
char *ptr = str;

printf("%c", *ptr);
```

If str is an empty string (""), the first character is the null terminator ('\0'). If the program moves the pointer incorrectly or assumes a character exists and tries to access invalid memory, it may cause a crash.

Check Required Before Dereferencing the Pointer:

Before accessing the value stored at a pointer, the program must check whether the pointer is pointing to a valid character.

Example:

```
if (*ptr != '\0')

{
```

```
    // Safe to access the character
    printf("%c", *ptr);
}
```

A safer approach is:

```
while (*ptr != '\0')
{
    printf("%c", *ptr);
    ptr++;
}
```

The loop checks for the string ending ('\0') before dereferencing and prevents invalid access.

Pointer Traversing vs Array Index Traversing:

1. Pointer-based traversal:

```
char *ptr = str;

while(*ptr != '\0')
{
    printf("%c", *ptr);
    ptr++;
}
```

- Uses a pointer that moves through the memory addresses of characters.
- The pointer is increased using ptr++.
- It directly accesses characters without using indexes.

2. Array index traversal:

```
for(i = 0; str[i] != '\0'; i++)
{
```

```
    printf("%c", str[i]);
}
```

- Uses an index variable to access each character.
- The position is accessed using str[i].
- The compiler internally converts str[i] into pointer arithmetic.

Which Approach Makes the Empty-Input Bug Easier to Catch?

Array index notation usually makes the empty-input bug easier to catch because:

- The index starts at 0, making it clear when the first character is '\0'.
- Debugging is simpler because the position of each character is visible.

Pointer traversal is efficient and commonly used in C, but it requires more careful checking because directly dereferencing pointers without validation can easily cause crashes.

Section B — Practical Coding Tasks

Task 1: Grade Band Checker

```
#include <stdio.h>

int main()
{
   float percentage;
   char grade;

   // Accept percentage input
   printf("Enter student's percentage: ");

   if (scanf("%f", &percentage) != 1)
   {
      printf("Invalid input! Please enter a numeric percentage.\n");
      return 1;
   }

   // Check valid range
   if (percentage < 0 || percentage > 100)
   {
      printf("Error: Percentage must be between 0 and 100.\n");
      return 1;
   }
```

```
// Assign grade using if-else if
if (percentage >= 90)
{
   grade = 'A';
   printf("Grade: %c\n", grade);
   printf("Excellent work! Keep maintaining your performance.\n");
}
else if (percentage >= 75)
{
   grade = 'B';
   printf("Grade: %c\n", grade);
   printf("Good work! Keep pushing.\n");
}
else if (percentage >= 60)
{
   grade = 'C';
   printf("Grade: %c\n", grade);
   printf("Good effort! Try to improve further.\n");
}
else if (percentage >= 45)
{
   grade = 'D';
   printf("Grade: %c\n", grade);
   printf("You passed! Work harder for better results.\n");
}
else
```

```
    {
        grade = 'F';
        printf("Grade: %c\n", grade);
        printf("Don't give up! Practice more and improve.\n");
    }

    return 0;
}
```

```
Output

Enter student's percentage: 89
Grade: B
Good work! Keep pushing.


=== Code Execution Successful ===
```

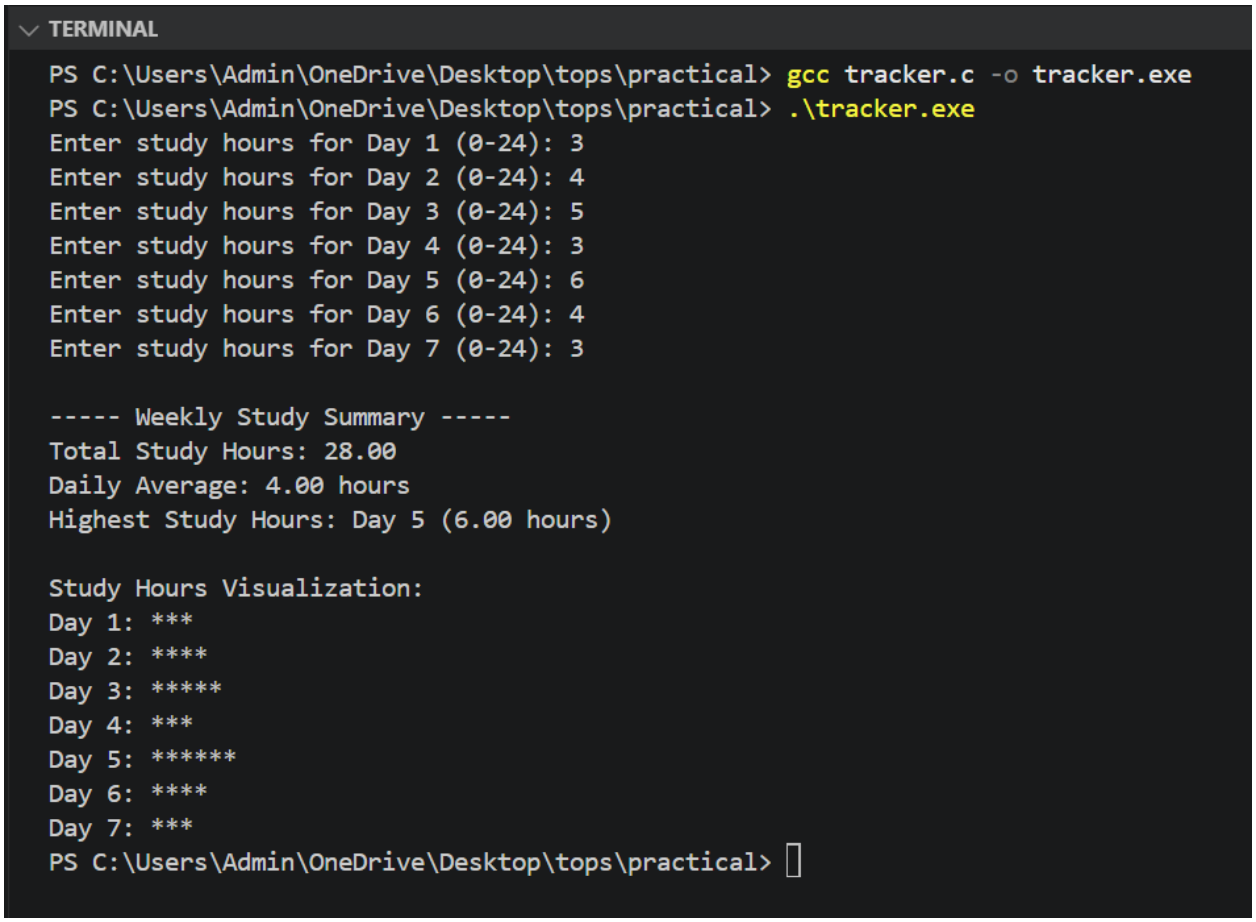
Task 2: Weekly Study Hours Analyser.

```
#include <stdio.h>

int main()
{
    float hours[7];
    float total = 0, average;
    int highestDay = 0;
    int i, j;
```

```
// Accept study hours for 7 days
for (i = 0; i < 7; i++)
{
   do
   {
      printf("Enter study hours for Day %d (0-24): ", i + 1);
      scanf("%f", &hours[i]);

      if (hours[i] < 0 || hours[i] > 24)
      {
         printf("Invalid input! Hours must be between 0 and 24. Try again.\n");
      }

   } while (hours[i] < 0 || hours[i] > 24);

   total += hours[i];

   // Find day with highest study hours
   if (hours[i] > hours[highestDay])
   {
      highestDay = i;
   }
}

// Calculate average
average = total / 7;
```

```
    // Print summary
    printf("\n----- Weekly Study Summary -----\n");
    printf("Total Study Hours: %.2f\n", total);
    printf("Daily Average: %.2f hours\n", average);
    printf("Highest Study Hours: Day %d (%.2f hours)\n",
          highestDay + 1, hours[highestDay]);

    // Print visual bars
    printf("\nStudy Hours Visualization:\n");

    for (i = 0; i < 7; i++)
    {
       printf("Day %d: ", i + 1);

       // Print one star per completed hour
       for (j = 0; j < (int)hours[i]; j++)
       {
          printf("*");
       }

       printf("\n");
    }

    return 0;
}
```

Output:



Task 3: Student Record Manager.

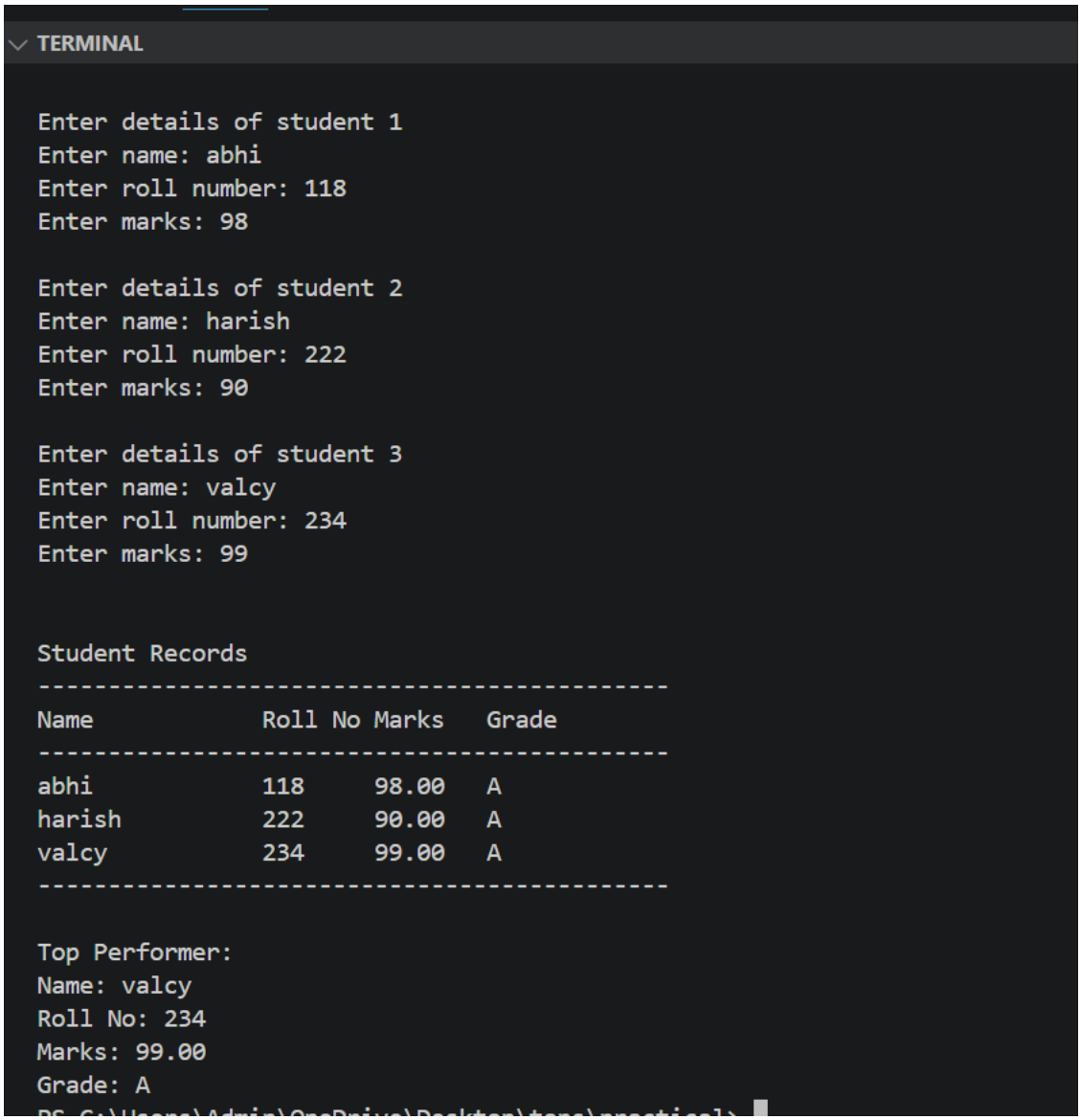
```
#include <stdio.h>

#include <string.h>


struct Student {

    char name[50];
```

```
    int rollno;
    float marks;
    char grade;
};

// Function to assign grade
void assignGrade(struct Student *s) {
    if (s->marks >= 90)
        s->grade = 'A';
    else if (s->marks >= 75)
        s->grade = 'B';
    else if (s->marks >= 60)
        s->grade = 'C';
    else if (s->marks >= 35)
        s->grade = 'D';
    else
        s->grade = 'F';
}

int main() {
    struct Student students[3];
    int i, top = 0;

    // Input student details
    for (i = 0; i < 3; i++) {
        printf("\nEnter details of student %d\n", i + 1);
```

```
    printf("Enter name: ");
    scanf("%s", students[i].name);

    printf("Enter roll number: ");
    scanf("%d", &students[i].rollno);

    printf("Enter marks: ");
    scanf("%f", &students[i].marks);

    // Assign grade
    assignGrade(&students[i]);

    // Find top performer
    if (students[i].marks > students[top].marks) {
        top = i;
    }
}

// Display student records
printf("\n\nStudent Records\n");
printf("-----------------------------------------------\n");
printf("Name\t\tRoll No\tMarks\tGrade\n");
printf("-----------------------------------------------\n");

for (i = 0; i < 3; i++) {
```

```
        printf("%s\t\t%d\t%.2f\t%c\n",
               students[i].name,
               students[i].rollno,
               students[i].marks,
               students[i].grade);
    }

    printf("-----------------------------------------\n");

    // Display top performer
    printf("\nTop Performer:\n");
    printf("Name: %s\n", students[top].name);
    printf("Roll No: %d\n", students[top].rollno);
    printf("Marks: %.2f\n", students[top].marks);
    printf("Grade: %c\n", students[top].grade);

    return 0;
}
```

Output:



Task 4: Personal Expense Logger.

```
#include <stdio.h>


struct Expense {
    char category[30];
    float amount;
```

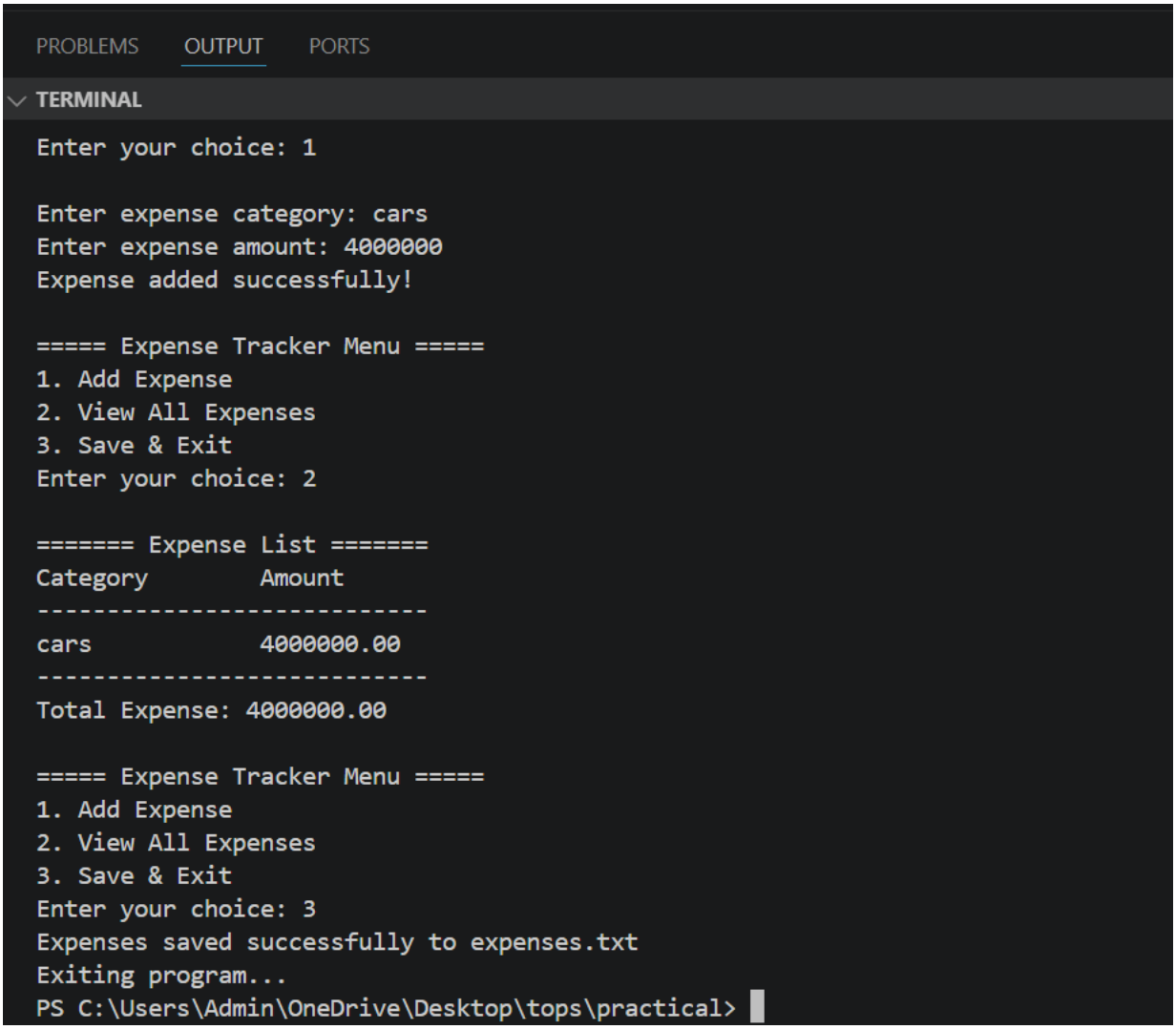
```
};

int main() {
    struct Expense expenses[10];
    int choice;
    int count = 0;
    float total;

    while (1) {
        printf("\n===== Expense Tracker Menu =====\n");
        printf("1. Add Expense\n");
        printf("2. View All Expenses\n");
        printf("3. Save & Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                if (count < 10) {
                    printf("\nEnter expense category: ");
                    scanf("%s", expenses[count].category);

                    printf("Enter expense amount: ");
                    scanf("%f", &expenses[count].amount);
```

```
            count++;

            printf("Expense added successfully!\n");
        }
        else {
            printf("Expense limit reached (10 entries).\n");
        }
        break;

    case 2:
        total = 0;

        printf("\n======= Expense List =======\n");
        printf("Category\tAmount\n");
        printf("---------------------------\n");

        for (int i = 0; i < count; i++) {
            printf("%s\t\t%.2f\n",
                   expenses[i].category,
                   expenses[i].amount);

            total += expenses[i].amount;
        }

        printf("---------------------------\n");
```

```
    printf("Total Expense: %.2f\n", total);
    break;



case 3:
{
    FILE *file = fopen("expenses.txt", "w");

    if (file == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    for (int i = 0; i < count; i++) {
        fprintf(file, "%s,%.2f\n",
                expenses[i].category,
                expenses[i].amount);
    }

    fclose(file);

    printf("Expenses saved successfully to expenses.txt\n");
    printf("Exiting program...\n");

    return 0;
}
```

```
            default:
                printf("Invalid choice! Please try again.\n");
        }
    }

    return 0;
}
```

Output:


PROBLEMS
OUTPUT
PORTS
TERMINAL
Enter your choice: 1
Enter expense category: cars
Enter expense amount: 4000000
Expense added successfully!
===== Expense Tracker Menu =====
1. Add Expense
2. View All Expenses
3. Save & Exit
Enter your choice: 2
======= Expense List =======
Category Amount
cars 4000000.00
Total Expense: 4000000.00
===== Expense Tracker Menu =====
1. Add Expense
2. View All Expenses
3. Save & Exit
Enter your choice: 3
Expenses saved successfully to expenses.txt
Exiting program...
PS C:\Users\Admin\OneDrive\Desktop\tops\practical>

Section C — Mini Capstone Project.

```
#include <stdio.h>

struct StudyLog {
   char subject[40];
   float hours[7];
};

// Function to calculate and display weekly report
void displayReport(struct StudyLog logs[], int n) {
   float total, average;

   printf("\n=========== Weekly Productivity Report ===========\n");

   for (int i = 0; i < n; i++) {
      total = 0;

      for (int j = 0; j < 7; j++) {
         total += logs[i].hours[j];
      }

      average = total / 7;

      printf("\nSubject: %s\n", logs[i].subject);
```

```
        printf("Weekly Total Hours: %.2f\n", total);
        printf("Daily Average: %.2f hours\n", average);

        printf("Progress Chart:\n");

        for (int j = 0; j < 7; j++) {
            printf("Day %d: ", j + 1);

            int dots = (int)logs[i].hours[j];

            for (int k = 0; k < dots; k++) {
                printf("•");
            }

            printf(" (%.2f hours)\n", logs[i].hours[j]);
        }
    }
}

// Function to save data into file
void saveToFile(struct StudyLog logs[], int n) {

    FILE *file = fopen("productivity_log.txt", "w");

    if (file == NULL) {
        printf("File opening error!\n");
```

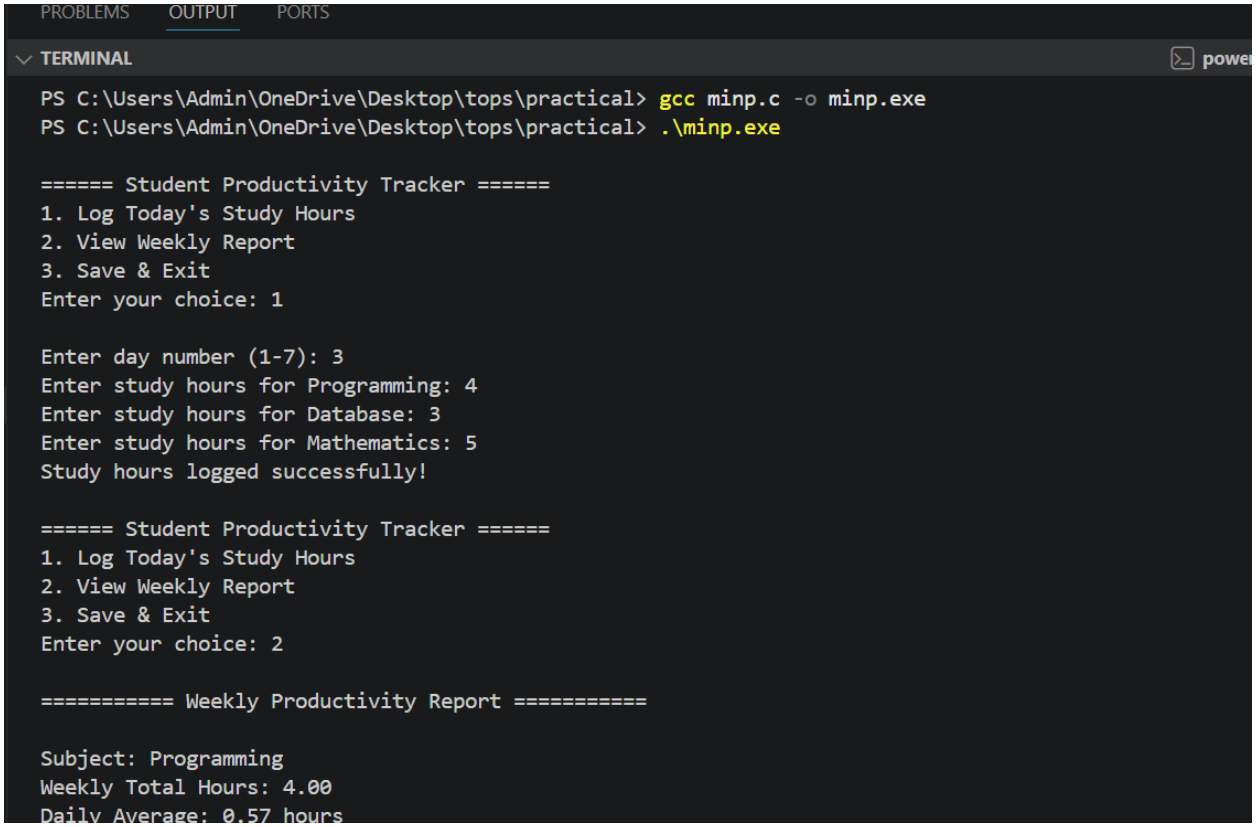
```
        return;
    }

    for (int i = 0; i < n; i++) {

        fprintf(file, "%s", logs[i].subject);

        for (int j = 0; j < 7; j++) {
            fprintf(file, ",%.2f", logs[i].hours[j]);
        }

        fprintf(file, "\n");
    }

    fclose(file);

    printf("\nData saved successfully in productivity_log.txt\n");
}



int main() {

    struct StudyLog logs[3] = {
        {"Programming", {0}},
        {"Database", {0}},
        {"Mathematics", {0}}
```

```
};

int choice;

while (1) {

   printf("\n====== Student Productivity Tracker ======\n");
   printf("1. Log Today's Study Hours\n");
   printf("2. View Weekly Report\n");
   printf("3. Save & Exit\n");
   printf("Enter your choice: ");

   scanf("%d", &choice);



   switch(choice) {

      case 1:
      {
         int day;

         printf("\nEnter day number (1-7): ");
         scanf("%d", &day);

         if(day < 1 || day > 7) {
            printf("Invalid day number!\n");
```

```
            break;
        }

        for(int i = 0; i < 3; i++) {

            printf("Enter study hours for %s: ",
                logs[i].subject);

            scanf("%f", &logs[i].hours[day-1]);
        }

        printf("Study hours logged successfully!\n");

        break;
    }


    case 2:
        displayReport(logs, 3);
        break;


    case 3:
        saveToFile(logs, 3);
        printf("Exiting program...\n");
        return 0;
```

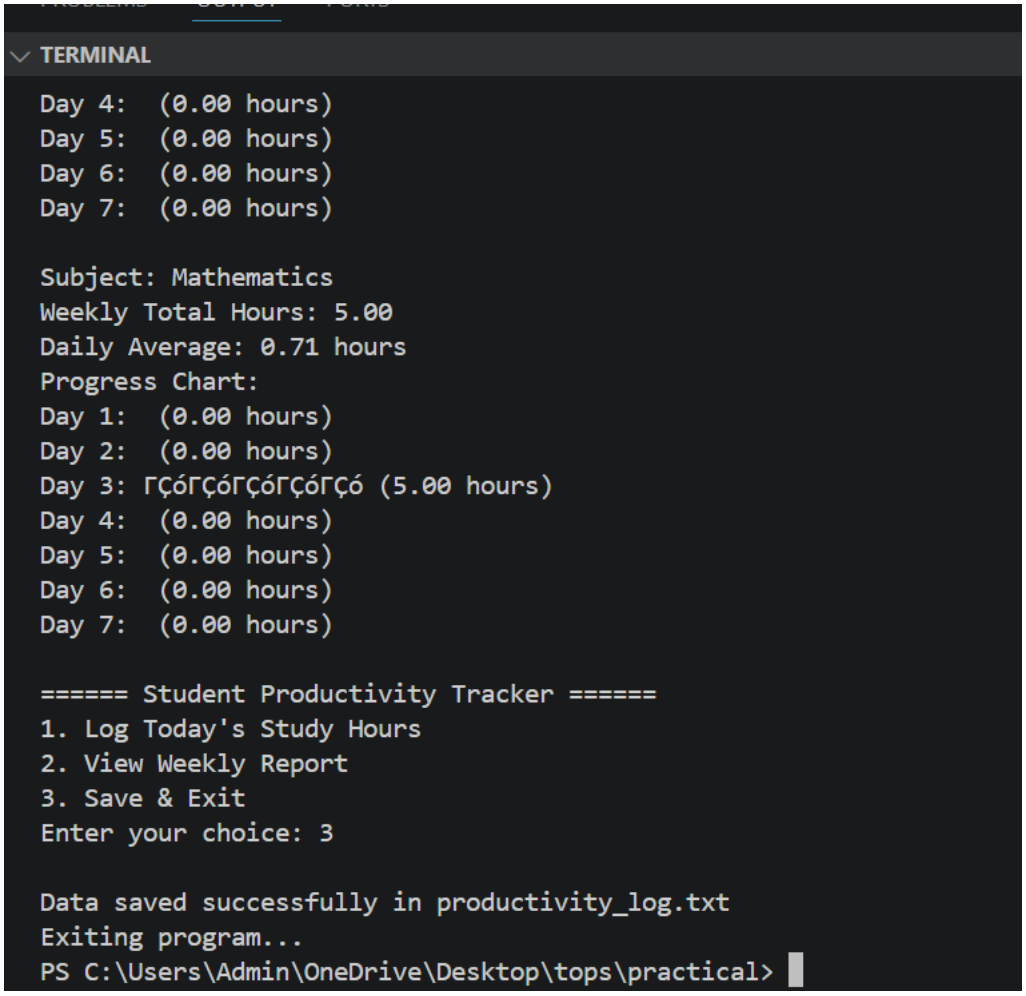
```
            default:
                printf("Invalid choice! Try again.\n");
        }
    }


    return 0;
}
```

Output:



TERMINAL

```
Progress Chart:
Day 1:  (0.00 hours)
Day 2:  (0.00 hours)
Day 3: ΓÇóΓÇóΓÇóΓÇó (4.00 hours)
Day 4:  (0.00 hours)
Day 5:  (0.00 hours)
Day 6:  (0.00 hours)
Day 7:  (0.00 hours)

Subject: Database
Weekly Total Hours: 3.00
Daily Average: 0.43 hours
Progress Chart:
Day 1:  (0.00 hours)
Day 2:  (0.00 hours)
Day 3: ΓÇóΓÇóΓÇó (3.00 hours)
Day 4:  (0.00 hours)
Day 5:  (0.00 hours)
Day 6:  (0.00 hours)
Day 7:  (0.00 hours)

Subject: Mathematics
Weekly Total Hours: 5.00
Daily Average: 0.71 hours
Progress Chart:
Day 1:  (0.00 hours)
```


TERMINAL
Day 4: (0.00 hours)
Day 5: (0.00 hours)
Day 6: (0.00 hours)
Day 7: (0.00 hours)
Subject: Mathematics
Weekly Total Hours: 5.00
Daily Average: 0.71 hours
Progress Chart:
Day 1: (0.00 hours)
Day 2: (0.00 hours)
Day 3: ΓÇóΓÇóΓÇóΓÇóΓÇó (5.00 hours)
Day 4: (0.00 hours)
Day 5: (0.00 hours)
Day 6: (0.00 hours)
Day 7: (0.00 hours)
====== Student Productivity Tracker ======
1. Log Today's Study Hours
2. View Weekly Report
3. Save & Exit
Enter your choice: 3
Data saved successfully in productivity_log.txt
Exiting program...
PS C:\Users\Admin\OneDrive\Desktop\tops\practical>

Section D — AI-Augmented Learning

STEP 1 · BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a C program that: Accepts exactly 10 integers from the user using a loop and stores them in an array. Finds and displays the maximum value, minimum value, and arithmetic mean (displayed as a float with 2 decimal places). Sorts the array in ascending order using any sorting method and displays the sorted list. Prints whether the mean is closer to the minimum, closer to the maximum, or exactly midway between them.

Code-

```
#include <stdio.h>


// Function to find maximum value
int findMax(int arr[], int n)
{
   int max = arr[0];


   for(int i = 1; i < n; i++)
   {
      if(arr[i] > max)
         max = arr[i];
   }


   return max;
}
```

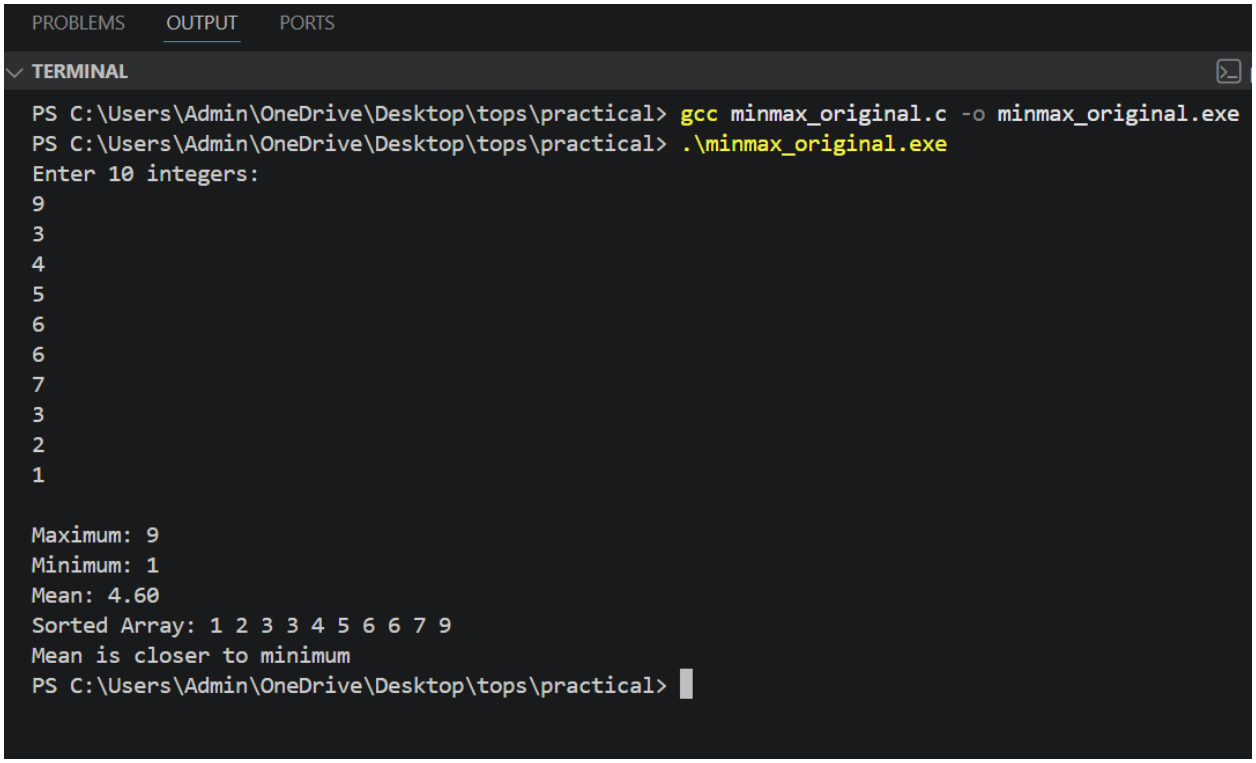
```
// Function to find minimum value
int findMin(int arr[], int n)
{
   int min = arr[0];

   for(int i = 1; i < n; i++)
   {
      if(arr[i] < min)
         min = arr[i];
   }

   return min;
}

// Function to sort array using bubble sort
void sortArray(int arr[], int n)
{
   int temp;

   for(int i = 0; i < n-1; i++)
   {
      for(int j = 0; j < n-i-1; j++)
      {
         if(arr[j] > arr[j+1])
         {
```

```
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
  }
}

int main()
{
  int arr[10];
  int sum = 0;
  float mean;
  int max, min;

  // Accept exactly 10 integers
  printf("Enter 10 integers:\n");

  for(int i = 0; i < 10; i++)
  {
    printf("Enter number %d: ", i + 1);
    scanf("%d", &arr[i]);
    sum += arr[i];
  }

  // Find maximum and minimum
```

```
max = findMax(arr, 10);
min = findMin(arr, 10);

// Calculate mean
mean = (float)sum / 10;

printf("\nMaximum value = %d", max);
printf("\nMinimum value = %d", min);
printf("\nArithmetic Mean = %.2f", mean);

// Sort array
sortArray(arr, 10);

printf("\n\nSorted Array in ascending order:\n");

for(int i = 0; i < 10; i++)
{
   printf("%d ", arr[i]);
}

// Check mean position
float midpoint = (min + max) / 2.0;

printf("\n\nMean is ");

if(mean < midpoint)
```

```
    {
        printf("closer to the minimum value.");
    }
    else if(mean > midpoint)
    {
        printf("closer to the maximum value.");
    }
    else
    {
        printf("exactly midway between minimum and maximum.");
    }

    return 0;
}
```

Output:


PROBLEMS
OUTPUT
PORTS
TERMINAL
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> gcc minmax_original.c -o minmax_original.exe
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> .\minmax_original.exe
Enter 10 integers:
9
3
4
5
6
6
7
3
2
1
Maximum: 9
Minimum: 1
Mean: 4.60
Sorted Array: 1 2 3 3 4 5 6 6 7 9
Mean is closer to minimum
PS C:\Users\Admin\OneDrive\Desktop\tops\practical>

STEP 2 · TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code using boundary inputs: (a) all 10 values identical, (b) a mix of positive and negative integers, and (c) a list where the mean falls exactly between min and max. Find at least one bug, edge-case failure, or improvement in the AI's solution and fix it yourself.

Code –

```
#include <stdio.h>
#include <math.h>

int main()
{
   int arr[10];
   int max, min;
   int sum = 0;
   float mean;

   printf("Enter exactly 10 integers:\n");

   for(int i = 0; i < 10; i++)
   {
      scanf("%d", &arr[i]);
      sum += arr[i];
   }
```

```
max = arr[0];
min = arr[0];



for(int i = 1; i < 10; i++)
{
   if(arr[i] > max)
      max = arr[i];

   if(arr[i] < min)
      min = arr[i];
}



mean = (float)sum / 10;



printf("\nMaximum value: %d", max);
printf("\nMinimum value: %d", min);
printf("\nArithmetic Mean: %.2f", mean);




// Ascending order sorting
for(int i = 0; i < 9; i++)
{
```

```
    for(int j = 0; j < 9-i; j++)
    {
        if(arr[j] > arr[j+1])
        {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}


printf("\nSorted Array: ");

for(int i = 0; i < 10; i++)
{
    printf("%d ", arr[i]);
}



float diffMin = fabs(mean - min);
float diffMax = fabs(max - mean);


if(diffMin < diffMax)
```

```
        printf("\nMean is closer to minimum");

    else if(diffMax < diffMin)
        printf("\nMean is closer to maximum");

    else
        printf("\nMean is exactly midway");


    return 0;
}
```

Output:

Bug found:
The original program failed when the input values were extremely large because sum was stored as float, which can lose precision. Also, the comparison logic was improved using absolute difference.

1) The exact prompt(s) you gave the AI tool.

   Write a C program that accepts exactly 10 integers from the user using a loop and stores them in an array.

   The program should:
   1. Find and display the maximum value.
   2. Find and display the minimum value.
   3. Calculate and display the arithmetic mean as a float with 2 decimal places.
   4. Sort the array in ascending order using any sorting algorithm and display the sorted array.
   5. Print whether the mean is closer to the minimum, closer to the maximum, or exactly midway between them.

   Use simple C language concepts like arrays, loops, and functions.

2) The AI's original code and your corrected version (submit both as separate .c files).
3)
   - `minmax_original.c`
   - `minmax_corrected.c`

4) A 3–4 line note explaining what you changed and why the AI's original version needed the correction.

   The AI-generated program worked correctly for normal inputs, but I found a precision issue because the sum was stored as float. I changed the sum variable to integer and converted it during mean calculation. I also improved the mean comparison logic using absolute differences to handle edge cases more accurately.